Academic Statement

Haoyu Li
lihaoyu499@gmail.com

University of Illinois Urbana-Champaign
PhD in Computer Science Program

My research interests are **software and system security**. Given the pervasive use of complex software systems, enhancing their reliability and security has become an imperative priority. Despite this urgency, accurately capturing the behavior of real-world programs and identifying security issues is highly challenging, due to the inherent complexities of programming languages and the varied coding practices of developers. With the development of AI technology, large language models (LLMs) have shown impressive potential in code analysis tasks, offering new opportunities to tackle these challenges. Motivated along these lines, **my goal as a researcher is to push the boundaries of automated security analysis** by combining cutting-edge program analysis techniques with the capabilities of LLMs. I believe that pursuing graduate studies in the Computer Science PhD program at the University of Illinois Urbana-Champaign, will help me achieve this goal.

Embedded System Security Research. My first research project focused on automated vulnerability detection for embedded systems, starting in 2022. Given that most IoT devices interact with users via network services, improper handling of user input data can allow attackers to craft malicious data and further compromise the device, leading to severe security issues. Under the guidance of my master's advisor, Libo Chen, I improved and extended his previous work [1, 2] to enable more effective vulnerability detection for both Linux-based and RTOS[1]-based firmware binaries. **My main contributions** were in the following two aspects. On the one hand, I addressed the data-sharing paradigms (such as environment variables and NVRAM[2]), which interrupt the taint propagation flow by storing and loading data from global regions. By concatenating control flows interrupted by data-sharing operations, we **reduced false negatives** and uncovered more complex vulnerability traces. On the other hand, I wrote a module to infer potential sizes of the heap and global buffer by performing backward reach definition analysis from parameters at sink points. This improvement allows for more precise buffer boundary checking and effectively **reduces false positives**. The optimized analysis framework enabled us to **discover more than 10 zero-day vulnerabilities in mainstream IoT devices**, which have been assigned CVE/CNVD IDs. Additionally, our work resulted in **a paper submitted to one of the top 2 software engineering journals** for review.

Turning to Research on More Complex Systems. In 2024, my research focus shifted to the analysis of more complex systems – specifically, **the Linux kernel**. The motivation stems from my experience in playing CTF[3] games, in which I found Linux kernel vulnerability exploitation to be fascinating and challenging. Can we strengthen kernel protection mechanisms or identify kernel bugs through an automated approach? Driven by this question, I shifted from firmware binary analysis to kernel source code analysis and decided to work on the following two research projects.

[1] Real Time Operate System
[2] Non-Volatile Random Access Memory
[3] CTF (Capture The Flag) is a type of cybersecurity competition.

Static Analyzer Generation based on Bug Patches. In the maintenance of Linux kernel code, though patches can fix specific bugs, similar bug patterns are likely present elsewhere in the large code base, which makes it **difficult to conduct thorough inspections manually**. In this work, our goal is to drive LLMs to extract bug patterns from provided patches and generate effective static analyzers. Under the guidance of Professor Lingming Zhang from the University of Illinois Urbana-Champaign, I collaborated with his two PhD students and another intern to design an automated pipeline for static analyzer generation. **My main contributions** were building the pipeline and embedding expert knowledge of the Clang Static Analyzer into it. I believe that my insights into program analysis have helped this project stay on the right track. So far, **we have detected 60+ previously unknown bugs, with more than 40 confirmed**. These results were derived by more than 20 generated static analyzers, covering nearly 10 bug types. For these reported bugs, I identified over a quarter of them independently. We expect to release a preprint[4] of this work by January 2025.

Inline Assembly Handling in C Code. For many source-level C code static analysis tools working on LLVM IR, inline assembly has become **a significant overlooked issue**, particularly in Linux kernel-related work. Because of **the inherent difficulty of cross-language analysis**, existing tools tend to either overlook this C language feature or handle it manually, resulting in unsound implementations. Under the guidance of Professor Zhiyun Qian at the University of California, Riverside, I began exploring this issue and **led the project independently**. To propose a universal and scalable solution, our insight is that LLMs can reason about the behavior of inline assembly code with their knowledge of instruction sets across various architectures. I designed a workflow to **guide LLMs in generating function summaries** that describe the program behaviors of interest, and implemented the prototype within the SVF framework [4]. In the Linux kernel global variable alias analysis and protection scenario [3], **our work eliminated 45 erroneous results**, i.e., global variables with write operations were mistakenly considered protectable[5] due to inline assembly.

Future Plans & Career Goals. In my future graduate studies, I aim to tackle challenging problems in the security analysis of complex software and systems. I am also passionate about leveraging the capabilities of LLMs to advance program analysis techniques at both the source and binary levels. **After completing my PhD**, I plan to engage in research that has a tangible impact on the real world. It is exciting to **design and build fundamental analysis tools or frameworks that can be applied in various scenarios**. In the long term, I hope to make a positive impact on the world through my research contributions as an academic.

Interest in UIUC. I am particularly drawn to the PhD program in Computer Science at the University of Illinois Urbana-Champaign, due to its world-class research, distinguished faculty, and collaborative environment. **Professor Luyi Xing**'s work on IoT system security inspires my research greatly, and we share an interest in AI-driven security. **Professor Tianyin Xu**'s research on system bug detection aligns with my experience and plans. I would also like to work with **Professor Darko Marinov**, whose expertise in software testing and bug detection resonates with my goals. Another potential advisor I am excited about working with is **Professor Gang Wang**, whose focus on enhancing human capabilities in security tasks overlaps with my objectives.

Overall, I believe that starting my PhD journey at UIUC will be a key step in achieving my research objectives.

---

[4]Mentioned preprints will be available at: https://learjet5.github.io/research/

[5]Global variables satisfying "read-only after init" are protectable.

References

[1] Libo Chen, Quanpu Cai, Zhenbang Ma, Yanhao Wang, Hong Hu, Minghang Shen, Yue Liu, Shanqing Guo, Haixin Duan, Kaida Jiang, et al. SFuzz: Slice-based Fuzzing for Real-Time Operating Systems. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 485–498, 2022.

[2] Libo Chen, Yanhao Wang, Quanpu Cai, Yunfan Zhan, Hong Hu, Jiaqi Linghu, Qinsheng Hou, Chao Zhang, Haixin Duan, and Zhi Xue. Sharing more and checking less: Leveraging Common Input Keywords to Detect Bugs in Embedded Systems. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 303–319, 2021.

[3] Guoren Li, Hang Zhang, Jinmeng Zhou, Wenbo Shen, Yulei Sui, and Zhiyun Qian. A Hybrid Alias Analysis and its Application to Global Variable Protection in the Linux Kernel. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 4211–4228, 2023.

[4] Yulei Sui and Jingling Xue. SVF: Interprocedural Static Value-Flow Analysis in LLVM. In *Proceedings of the 25th international conference on compiler construction*, pages 265–266. ACM, 2016.